

Consumer Shopping Behavior Analysis

1. Project Overview

Business Problem Statement

A leading retail company wants to better understand its customers' shopping behavior in order to improve sales, customer satisfaction, and long-term loyalty. The management team has noticed changes in purchasing patterns across demographics, product categories, and sales channels. They are particularly interested in uncovering which factors — such as discounts, reviews, seasons, or payment preferences — drive consumer decisions and repeat purchases.

Central Business Question

“How can the company leverage consumer shopping data to identify trends, improve customer engagement, and optimize marketing and product strategies?”

Objectives

- Analyse revenue and spending patterns across gender, age, and subscription segments.
- Identify the impact of discounts and promo codes on purchase amounts.
- Evaluate product performance through review ratings.
- Compare shipping preferences and their relationship to spending behaviour.
- Build an interactive Power BI dashboard to enable self-service business insights.

Tools & Technologies

Tool	Purpose
Python (Pandas)	Data cleaning, feature engineering, and exploratory analysis
PostgreSQL / SQL	Business queries and transactional analysis
Power BI	Interactive dashboard and visual reporting

2. Dataset Summary

The dataset contains transactional and demographic records of retail customers. Below is an overview of the key characteristics.

Metric	Value	Note
Total Records	3,900	After data cleaning
Total Features	18 (original) → 17	Promo Code Used dropped
Age Range	18 – 70 years	4 age group bins created
Gender	Male, Female	2 categories
Product Categories	4	Clothing, Footwear, Outerwear, Accessories
Seasons	4	Winter, Spring, Summer, Fall
Payment Methods	6	Credit Card, Cash, Venmo, PayPal, Bank Transfer, Debit Card
Purchase Amount Range	\$20 – \$100	USD
Average Review Rating	3.75 / 5	Post-imputation
Missing Values	37 (Review Rating)	Imputed using category-level median

Column Reference

Column Name	Type	Description
Customer ID	<i>int</i>	Unique identifier for each customer
Age	<i>int</i>	Customer's age (18–70)
Gender	<i>string</i>	Male / Female
Item Purchased	<i>string</i>	Name of the product purchased
Category	<i>string</i>	Product category (Clothing, Footwear, Outerwear, Accessories)
Purchase Amount (USD)	<i>int</i>	Transaction value in USD (20–100)
Location	<i>string</i>	State/region of the customer
Size	<i>string</i>	Product size selected
Color	<i>string</i>	Product colour selected
Season	<i>string</i>	Season of the transaction (Winter/Spring/Summer/Fall)
Review Rating	<i>float</i>	Customer review score (1–5 scale)

Subscription Status	<i>string</i>	Whether customer holds an active subscription
Shipping Type	<i>string</i>	Shipping method chosen
Discount Applied	<i>string</i>	Whether a discount was applied (Yes/No)
Promo Code Used	<i>string</i>	Whether a promo code was used (dropped – identical to Discount Applied)
Previous Purchases	<i>int</i>	Number of prior purchases by the customer
Payment Method	<i>string</i>	Payment channel (Credit Card, Cash, PayPal, Venmo, etc.)
Frequency of Purchases	<i>string</i>	How often the customer shops (Weekly, Monthly, etc.)

3. Exploratory Data Analysis using Python

The following steps were performed in the Jupyter Notebook (Customer_behavior_data_cleaning.ipynb) to clean and prepare the dataset for analysis.

1. Data Loading & Initial Inspection

Loaded the dataset using `pd.read_csv()`. Used `df.head()`, `df.info()`, and `df.describe()` to inspect shape, column data types, and basic statistics. Dataset contains 3,900 rows and 18 columns.

Code Screenshot:

[4]:

import pandas as pd

df = pd.read_csv('C:/Users/Admin/Downloads/Data Analysis Project/customer_shopping_behavior.csv')

[5]:

df.head()

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Shipping Type	Discount Applied	Promo Code Used	Previous Purchases	Payment Method
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray	Winter	3.1	Yes	Express	Yes	Yes	14	V
1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon	Winter	3.1	Yes	Express	Yes	Yes	2	
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	Yes	Free Shipping	Yes	Yes	23	
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon	Spring	3.5	Yes	Next Day Air	Yes	Yes	49	I
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turquoise	Spring	2.7	Yes	Free Shipping	Yes	Yes	31	I

[6]:

df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 18 columns):
Column Non-Null Count Dtype
--- --- -
0 Customer ID 3900 non-null int64
1 Age 3900 non-null int64
2 Gender 3900 non-null object
3 Item Purchased 3900 non-null object
4 Category 3900 non-null object
5 Purchase Amount (USD) 3900 non-null int64
6 Location 3900 non-null object
7 Size 3900 non-null object
8 Color 3900 non-null object
9 Season 3900 non-null object
10 Review Rating 3863 non-null float64
11 Subscription Status 3900 non-null object
12 Shipping Type 3900 non-null object
13 Discount Applied 3900 non-null object
14 Promo Code Used 3900 non-null object
15 Previous Purchases 3900 non-null int64
16 Payment Method 3900 non-null object
17 Frequency of Purchases 3900 non-null object
dtypes: float64(1), int64(4), object(13)
memory usage: 548.6+ KB

```
[7]: df.describe()
```

```
[7]:
```

	Customer ID	Age	Purchase Amount (USD)	Review Rating	Previous Purchases
count	3900.000000	3900.000000	3900.000000	3863.000000	3900.000000
mean	1950.500000	44.068462	59.764359	3.750065	25.351538
std	1125.977353	15.207589	23.685392	0.716983	14.447125
min	1.000000	18.000000	20.000000	2.500000	1.000000
25%	975.750000	31.000000	39.000000	3.100000	13.000000
50%	1950.500000	44.000000	60.000000	3.800000	25.000000
75%	2925.250000	57.000000	81.000000	4.400000	38.000000
max	3900.000000	70.000000	100.000000	5.000000	50.000000

2. Missing Value Treatment

Identified 37 null values in the Review Rating column. Imputed missing values using the median rating within each product Category group to preserve category-level rating distributions.

Output / Result:

```
8]: df.isnull().sum()
```

```
8]: Customer ID      0
Age                0
Gender             0
Item Purchased     0
Category           0
Purchase Amount (USD) 0
Location           0
Size              0
Color             0
Season            0
Review Rating      37
Subscription Status 0
Shipping Type      0
Discount Applied   0
Promo Code Used    0
Previous Purchases 0
Payment Method     0
Frequency of Purchases 0
dtype: int64
```

```
[9]: df['Review Rating'] = df.groupby('Category')['Review Rating'].transform(lambda x: x.fillna(x.median()))
```

```
[10]: df.isnull().sum()
```

```
[10]: Customer ID      0
Age                0
Gender             0
Item Purchased     0
Category           0
Purchase Amount (USD) 0
Location           0
Size              0
Color             0
Season            0
Review Rating      0
Subscription Status 0
Shipping Type      0
Discount Applied   0
Promo Code Used    0
Previous Purchases 0
Payment Method     0
Frequency of Purchases 0
dtype: int64
```

3. Column Standardisation

Renamed all columns to lowercase with underscores using `str.lower()` and `str.replace()`. Renamed 'purchase_amount_(usd)' to 'purchase_amount' for cleaner SQL and downstream analysis.

Code Screenshot:

```
[11]: df.columns = df.columns.str.lower()
      df.columns = df.columns.str.replace(' ', '_')
      df = df.rename(columns = {'purchase_amount_(usd)': 'purchase_amount'})

[12]: df.columns

[12]: Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
          'purchase_amount', 'location', 'size', 'color', 'season',
          'review_rating', 'subscription_status', 'shipping_type',
          'discount_applied', 'promo_code_used', 'previous_purchases',
          'payment_method', 'frequency_of_purchases'],
          dtype='object')
```

4. Feature Engineering – Age Group

Created a new categorical column `age_group` using `pd.qcut()` with four equal-frequency bins: Young Adult, Adult, Middle-aged, and Senior. This enables demographic segmentation in analysis.

Code Screenshot:

```
[13]: # create a column age_group
      labels = ['Young Adult', 'Adult', 'Middle-aged', 'Senior']
      df['age_group'] = pd.qcut(df['age'], q=4, labels = labels)

[14]: df[['age', 'age_group']].head(10)
```

	age	age_group
0	55	Middle-aged
1	19	Young Adult
2	50	Middle-aged
3	21	Young Adult
4	45	Middle-aged
5	46	Middle-aged
6	63	Senior
7	27	Young Adult
8	26	Young Adult
9	57	Middle-aged

5. Feature Engineering – Purchase Frequency (Days)

Mapped the Frequency of Purchases text values to numeric days using a dictionary (e.g., Weekly → 7, Monthly → 30, Annually → 365), creating a new column `purchase_frequency_days` for quantitative comparison.

Code Screenshot:

```
[15]: # create a column purchase_frequency_days
      frequency_mapping = {
          'Fortnightly': 14,
          'Weekly': 7,
          'Monthly': 30,
          'Quarterly': 90,
          'Bi-Weekly': 14,
          'Annually': 365,
          'Every 3 Months': 90
      }

      df['purchase_frequency_days'] = df['frequency_of_purchases'].map(frequency_mapping)
```

```
[16]: df[['purchase_frequency_days', 'frequency_of_purchases']].head(10)
```

```
[16]:
```

	purchase_frequency_days	frequency_of_purchases
0	14	Fortnightly
1	14	Fortnightly
2	7	Weekly
3	7	Weekly
4	365	Annually
5	7	Weekly
6	90	Quarterly
7	7	Weekly
8	365	Annually
9	90	Quarterly

6. Duplicate Column Removal

Verified that the Discount Applied and Promo Code Used columns were fully identical across all rows. Dropped Promo Code Used as a redundant feature, reducing the dataset to 17 columns.

Code Screenshot:

```
[17]: df[['discount_applied', 'promo_code_used']].head(10)
```

```
[17]:
```

	discount_applied	promo_code_used
0	Yes	Yes
1	Yes	Yes
2	Yes	Yes
3	Yes	Yes
4	Yes	Yes
5	Yes	Yes
6	Yes	Yes
7	Yes	Yes
8	Yes	Yes
9	Yes	Yes

```
[18]: (df['discount_applied'] == df['promo_code_used']).all()
```

```
[18]: np.True_
```

```
[19]: df = df.drop('promo_code_used', axis = 1)
```

```
[20]: df.columns
```

```
[20]: Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',  
        'purchase_amount', 'location', 'size', 'color', 'season',  
        'review_rating', 'subscription_status', 'shipping_type',  
        'discount_applied', 'previous_purchases', 'payment_method',  
        'frequency_of_purchases', 'age_group', 'purchase_frequency_days'],  
       dtype='object')
```


7. Data Export to PostgreSQL

Connected to a local PostgreSQL instance using SQLAlchemy and psycopg2. Exported the cleaned DataFrame into a table named 'customer' in the customer_behavior database using df.to_sql().

Code Screenshot:

```
[32]: from sqlalchemy import create_engine

# step 1: Connect to PostgreSQL
username = 'postgres'
password = '1234'
host = 'localhost'
port = '5432'
database = 'customer_behavior'

engine = create_engine(f"postgresql+psycopg2://{username}:{password}@{host}:{port}/{database}")

# step 2: Load database into PostgreSQL
table_name = "customer"
df.to_sql(table_name, engine, if_exists = "replace", index = False)

print(f>Data Successfully Loaded into table '{table_name}' in database '{database}'.")

Data Successfully Loaded into table 'customer' in database 'customer_behavior'.
```

4. Data Analysis using SQL (Business Transactions)

After cleaning and exporting the dataset to PostgreSQL, the following business questions were answered using structured SQL queries.

Q1: Total Revenue by Gender

Calculates total revenue generated by male vs. female customers using SUM() grouped by gender.

SQL Query:

```
--Q1. What is the total revenue generated by male vs. female customers?  
select gender, SUM(purchase_amount) as revenue  
from customer  
group by gender
```

Query Output:

	gender text	revenue numeric
1	Female	75191
2	Male	157890

Q2: Discount Users Above Average Spend

Identifies customers who used a discount but still spent more than the average purchase amount using a subquery with AVG().

SQL Query:

```
--Q2. Which customers used a discount but still spent more than the average purchase amount?  
select customer_id, purchase_amount  
from customer  
where discount_applied = 'Yes' and purchase_amount >= (select AVG(purchase_amount) from customer)
```

Query Output:

	customer_id bigint	purchase_amount bigint
1	2	64
2	3	73
3	4	90
4	7	85
5	9	97
6	12	68
7	13	72
8	16	81
9	20	90
10	22	62

Q3: Top 5 Products by Average Review Rating

Retrieves the top 5 products with the highest average review rating using ROUND(AVG()) and ORDER BY with LIMIT.

SQL Query:

```
-- Q3. Which are the top 5 products with the highest average review rating?
select item_purchased, ROUND(AVG(review_rating::numeric),2) as "Average Product Rating"
from customer
group by item_purchased
order by AVG(review_rating) desc
limit 5|
```

Query Output:

	item_purchased text	Average Product Rating numeric
1	Gloves	3.86
2	Sandals	3.84
3	Boots	3.82
4	Hat	3.80
5	Skirt	3.78

Q4: Standard vs. Express Shipping – Avg Purchase

Compares average purchase amounts between Standard and Express shipping types using AVG() filtered by shipping_type.

SQL Query:

```
--Q4. Compare the average Purchase Amounts between Standard and Express Shipping.
select shipping_type,
ROUND(AVG(purchase_amount),2)
from customer
where shipping_type in ('Standard', 'Express')
group by shipping_type
```

Query Output:

	shipping_type text	round numeric
1	Standard	58.46
2	Express	60.48

Q5: Subscriber vs. Non-Subscriber Spend Analysis

Compares average spend and total revenue between subscribed and non-subscribed customers to determine if loyalty drives higher spending.

SQL Query:

```
--Q5. Do subscribed customers spend more? Compare average spend and total revenue
--between subscribers and non-subscribers.
select subscription_status,
COUNT(customer_id) AS total_customers,
ROUND(AVG(purchase_amount),2) AS avg_spend,
ROUND(SUM(purchase_amount),2) AS total_revenue
FROM customer
GROUP BY subscription_status
ORDER BY total_revenue,avg_spend DESC;
```

Query Output:

	subscription_status text	total_customers bigint	avg_spend numeric	total_revenue numeric
1	Yes	1053	59.49	62645.00
2	No	2847	59.87	170436.00

Q6: Top 5 Products by Discount Rate

Identifies the 5 products with the highest percentage of purchases where a discount was applied, using a CASE-based SUM divided by COUNT to calculate the discount rate per product.

SQL Query:

```
--Q6. Which 5 products have the highest percentage of purchases with discounts applied?
SELECT item_purchased,
       ROUND(100.0 * SUM(CASE WHEN discount_applied = 'Yes' THEN 1 ELSE 0 END)/COUNT(*),2) AS discount_rate
FROM customer
GROUP BY item_purchased
ORDER BY discount_rate DESC
LIMIT 5;
```

Query Output:

	item_purchased text	discount_rate numeric
1	Hat	50.00
2	Sneakers	49.66
3	Coat	49.07
4	Sweater	48.17
5	Pants	47.37

Q7: Customer Segmentation – New, Returning, Loyal

Segments customers into New (1 previous purchase), Returning (2–10 purchases), and Loyal (>10 purchases) using a CASE expression inside a CTE, then counts the number of customers in each segment.

SQL Query:

```
--Q7. Segment customers into New, Returning, and Loyal based on their total
-- number of previous purchases, and show the count of each segment.
with customer_type as (
SELECT customer_id, previous_purchases,
CASE
  WHEN previous_purchases = 1 THEN 'New'
  WHEN previous_purchases BETWEEN 2 AND 10 THEN 'Returning'
  ELSE 'Loyal'
END AS customer_segment
FROM customer)

select customer_segment, count(*) AS "Number of Customers"
from customer_type
group by customer_segment;
```

Query Output:

	customer_segment text	Number of Customers bigint
1	Loyal	3116
2	New	83
3	Returning	701

Q8: Top 3 Most Purchased Products per Category

Uses a CTE with ROW_NUMBER() OVER (PARTITION BY category) to rank products by order count within each category and returns only the top 3 per category.

SQL Query:

```
--Q8. What are the top 3 most purchased products within each category?
WITH item_counts AS (
    SELECT category,
           item_purchased,
           COUNT(customer_id) AS total_orders,
           ROW_NUMBER() OVER (PARTITION BY category ORDER BY COUNT(customer_id) DESC) AS item_rank
    FROM customer
    GROUP BY category, item_purchased
)
SELECT item_rank, category, item_purchased, total_orders
FROM item_counts
WHERE item_rank <= 3;
```

Query Output:

	item_rank bigint	category text	item_purchased text	total_orders bigint
1	1	Accessori...	Jewelry	171
2	2	Accessori...	Sunglasses	161
3	3	Accessori...	Belt	161
4	1	Clothing	Blouse	171
5	2	Clothing	Pants	171
6	3	Clothing	Shirt	169
7	1	Footwear	Sandals	160
8	2	Footwear	Shoes	150
9	3	Footwear	Sneakers	145
10	1	Outerwear	Jacket	163
11	2	Outerwear	Coat	161

Q9: Repeat Buyers and Subscription Likelihood

Filters customers with more than 5 previous purchases and groups them by subscription status to determine whether repeat buyers are more likely to subscribe.

SQL Query:

```
--Q9. Are customers who are repeat buyers (more than 5 previous purchases) also likely to subscribe?
SELECT subscription_status,
       COUNT(customer_id) AS repeat_buyers
FROM customer
WHERE previous_purchases > 5
GROUP BY subscription_status;
```

Query Output:

	subscription_status text	repeat_buyers bigint
1	No	2518
2	Yes	958

Q10: Revenue Contribution by Age Group

Calculates total revenue contributed by each age group (Young Adult, Adult, Middle-aged, Senior) using SUM(purchase_amount), ordered by revenue in descending order.

SQL Query:

```
--Q10. What is the revenue contribution of each age group?
SELECT
    age_group,
    SUM(purchase_amount) AS total_revenue
FROM customer
GROUP BY age_group
ORDER BY total_revenue desc;
```

Query Output:

	age_group text	total_revenue numeric
1	Young Adult	62143
2	Middle-aged	59197
3	Adult	55978
4	Senior	55763

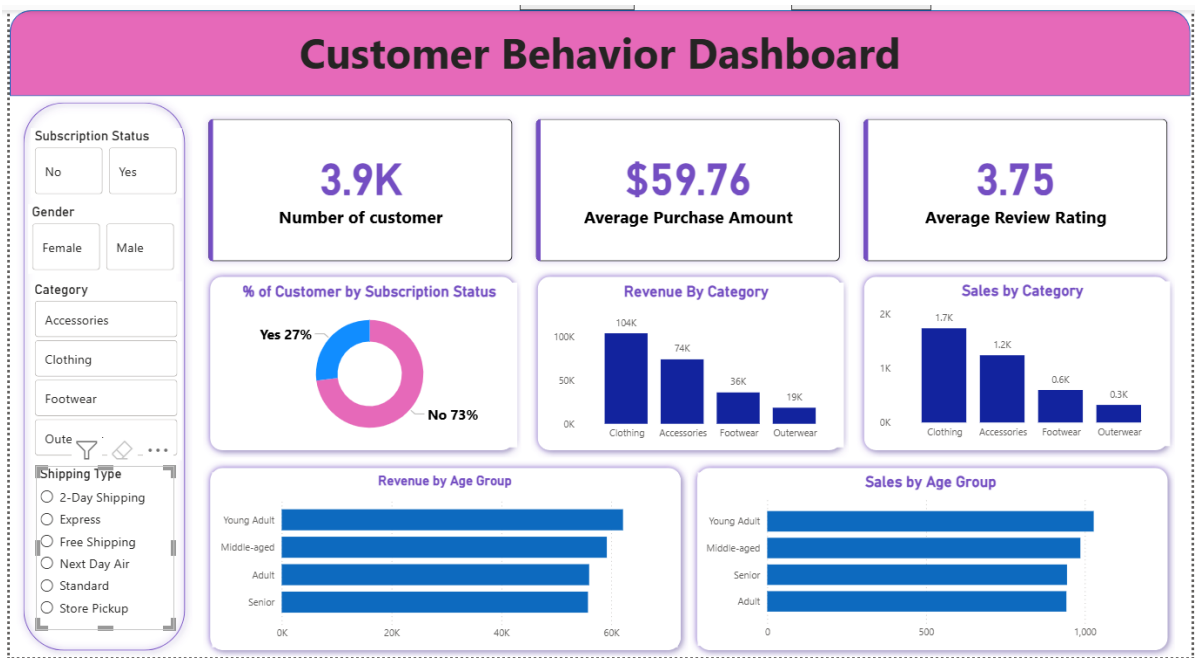
5. Dashboard in Power BI

An interactive dashboard was developed in Power BI using the cleaned dataset to provide a comprehensive, self-service view of customer shopping behaviour for business stakeholders.

Dashboard Overview

The Power BI report consolidates all key metrics and dimensions into a single-page interactive dashboard. Slicers allow filtering by Gender, Season, Category, and Subscription Status to enable dynamic exploration of the data.

Full Dashboard Screenshot:



Key Visuals Included

Visual	Insight it Provides
Revenue by Gender	Comparative total spend split between male and female customers
Revenue by Category	Identifies top-performing product categories by total revenue
Revenue by Season	Highlights seasonal purchase peaks for inventory planning
Top Products by Rating	Surfaces best-reviewed products for marketing prioritisation
Shipping Type Analysis	Compares average spend across shipping methods
Subscriber vs. Non-Subscriber	Evaluates loyalty programme ROI through spend comparison
Payment Method Distribution	Shows preferred payment channels across the customer base
Age Group Analysis	Reveals purchasing behaviour across life-stage segments

6. Business Recommendations

Based on the analysis conducted through Python, SQL, and the Power BI dashboard, the following recommendations are proposed to the management team.

1. Targeted Discount Strategy

Since discounted customers still outspend the average, the company should deploy targeted discount campaigns for high-intent segments rather than blanket promotions. This preserves margin while rewarding engaged buyers.

2. Subscription Programme Expansion

Subscribers demonstrate higher average spend and greater lifetime revenue. Investing in onboarding campaigns and subscriber-exclusive perks will increase conversion from casual to loyal customers.

3. Product & Category Prioritisation

Focus marketing budgets on the top-rated products identified in the review analysis. High-rated items should be featured in homepage placements and recommendation engines to drive conversion.

4. Seasonal Inventory & Campaign Planning

Align procurement and promotional calendars with seasonal purchase peaks identified in the dashboard. Pre-loading inventory for high-demand seasons reduces stockouts and improves customer satisfaction scores.

5. Gender-Specific Marketing

Revenue splits between male and female customers point to distinct purchasing preferences. Personalised campaigns, category emphasis, and payment options should be tailored per gender segment to maximise engagement.

6. Shipping Experience Optimisation

The comparison between Standard and Express shipping spend suggests an opportunity to upsell faster delivery at checkout. A/B testing shipping upgrade prompts for high-value carts can increase average order value.

End of Report